Cross-Platform WebXR Interaction Performance Comparison Between Meta Quest 3 and Apple Vision Pro

Nor Hady - 0523195
TU Berlin
Computer Science
Berlin, Germany
nor.hady@campus.tu-berlin.de

Xiaojie Sun - 0395832
TU Berlin
Computer Science
Berlin, Germany
lh7-summary@win.tu-berlin.de

***Abstract*—This paper presents a cross-platform performance and interaction study of a WebXR application deployed on the Meta Quest 3 and Apple Vision Pro. The goal is to evaluate whether a single browser-based XR application can support consistent interaction methods, real-time performance, and comparable user experiences across devices with fundamentally different input paradigms. To investigate this, we developed Star Catcher XR, a simple interactive game designed to measure interaction speed, accuracy, and responsiveness in a controlled environment. The system was implemented using Babylon.js and the WebXR Device API, with a modular architecture separating platform-independent interaction logic from device-specific input handling. This design enables direct comparison of interaction techniques, including controller-based raycasting and hand tracking on Meta Quest, and gaze-based selection with pinch gestures on Apple Vision Pro. Experimental results show that Meta Quest 3 provides stable and accurate interaction, with consistent performance and high usability. In contrast, interaction on Apple Vision Pro could not be fully realized due to limitations in gesture detection and gaze stability, restricting the evaluation to partial performance observations. While both devices achieved acceptable frame rates, differences in interaction reliability significantly impacted overall user experience. The findings demonstrate that WebXR is a viable foundation for cross-platform XR development, but highlight challenges in achieving consistent interaction across heterogeneous hardware. Future work should focus on improving interaction robustness and conducting broader evaluations with more participants to validate these results.**

***Index Terms*—WebXR, Extended Reality, Meta Quest 3, Apple Vision Pro, cross-platform interaction, performance evaluation**

I. Introduction

Extended Reality (XR) technologies are increasingly accessible through consumer devices such as the Meta Quest 3 and Apple Vision Pro. While these devices differ significantly in hardware design and interaction paradigms, WebXR provides a standardized browser-based interface for immersive experiences across platforms **(source)**.

The primary objective of this project is to investigate whether a single WebXR application can:

1) Run on both Meta Quest 3 and Apple Vision Pro,
2) Support all available interaction methods,
3) Enable real-time interaction,
4) Provide measurable performance and usability comparison.

To address this, we designed and implemented a browser-based XR experience called *Star Catcher XR*. The application places users in a shared 3D environment where they collect virtual stars within a 60-second time limit. The simplicity of the game mechanic allows focus on interaction efficiency and performance metrics rather than complex gameplay systems.

This paper describes the system architecture, interaction design, performance measurement framework, and comparative evaluation between the two devices.

II. System Design and Architecture

A. Development Framework

The application was implemented using Babylon.js, which provides a high-level rendering engine with built-in support for WebXR. This choice allowed rapid development of immersive functionality while maintaining full control over scene management and interaction logic. The WebXR Device API is used to initialize immersive sessions and access the available input sources for each device.

The entire system is deployed as a web application and runs directly in the browser. This deployment strategy reflects the core motivation of the project, namely the ability to distribute XR experiences without platform-specific installation processes. JavaScript ES modules were used to structure the application into reusable components. This modular architecture was particularly important for separating platform-independent interaction logic from device-specific input handling.

To enable real-time testing on standalone XR devices, the application was served using a local development server powered by Vite. Since immersive WebXR sessions require a secure HTTPS context, a tunneling service (ngrok) was used to expose the local server to the public internet. This allowed both Meta Quest and Apple Vision Pro to access the same

application instance via a secure URL.

This setup ensured that the exact same build of the application was tested on both devices, eliminating inconsistencies caused by different deployment environments. In addition, it enabled rapid iteration during development, as code changes could be immediately reflected on both devices without manual redeployment.

B. Environment Design

The virtual environment was intentionally designed to be visually simple in order to reduce performance variability between devices. A minimal scene consisting of a ground plane, a skybox, and dynamic lighting provides sufficient spatial orientation without introducing unnecessary rendering complexity. Interactive objects are spawned within a limited radius around the user to ensure that they remain within comfortable reach for both controller-based and gaze-based interaction.

This design decision serves two purposes. First, it ensures ergonomic interaction and reduces physical strain during repeated selection tasks. Second, it allows performance measurements to focus on input and interaction rather than GPU load caused by complex geometry or large textures.

C. Game Mechanics

The implemented experience follows a structured time-based interaction loop. At the beginning of each round, a timer is started, and interactive objects are spawned around the player. The user attempts to collect as many stars as possible within the time limit. Each successful interaction increases the score, while interacting with a bomb results in a penalty. After the timer expires, the interaction is disabled and the final score is displayed.

This game structure was chosen because it creates a repeatable and measurable interaction scenario. Since the same actions are performed multiple times within a fixed time interval, metrics such as reaction time, selection accuracy, and interaction speed can be compared across devices.

D. System Architecture

The application is divided into several modules with clearly defined responsibilities. The main initialization script creates the Babylon.js scene, loads assets, and starts the WebXR session. A shared grabbing system implements the core interaction logic, including raycasting, object selection, score updates, and visual feedback. Device-specific modules map the native input methods of each platform to this shared grabbing logic.

This separation between platform-independent interaction and platform-dependent input handling is essential for cross-platform development. It ensures that the game mechanics remain identical on both devices while allowing each input method to be implemented according to its technical requirements.

III. Interaction Design

A. Meta Quest 3

On the Meta Quest 3, the primary interaction method is controller-based raycasting. A virtual ray is projected from the controller into the scene, allowing the user to point at distant objects and select them using the trigger button. This technique provides high precision and fast response times because the physical controller offers stable tracking and tactile feedback. Haptic feedback further improves interaction confidence by confirming successful selections.

In addition to controllers, the Quest also supports optical hand tracking. In this mode, a pinch gesture is used to trigger object selection. Although this interaction feels more natural and reduces the need for physical controllers, it introduces slightly higher latency and reduced precision due to the limitations of vision-based gesture recognition.

B. Apple Vision Pro

Apple Vision Pro uses a fundamentally different interaction model based on gaze and pinch gestures. A continuous ray is projected from the user's head direction, and objects that intersect with this ray are highlighted. Selection is performed by a pinch gesture. This interaction paradigm is highly intuitive because it closely resembles natural human behavior, where attention is directed by looking at an object and performing a small hand gesture.

However, this approach also introduces new challenges. The accuracy of selection depends heavily on the alignment between gaze and hand gesture, and repeated rapid interactions can be slower compared to controller-based input.

C. Interaction Abstraction

To ensure consistent gameplay across platforms, all interaction methods are mapped to a shared grabbing event. This abstraction layer allows the game logic to remain unchanged regardless of whether the input originates from a controller, a hand gesture, or a gaze-based selection. As a result, differences in performance can be attributed to the hardware and tracking systems rather than differences in the application logic.

IV. Implementation

This section will go more in depth with the system achitechture and how the actual application was implementet more in depth.

A. Application Structure

The entry point of the application is `main.js`, which is responsible for initializing the Babylon.js scene, loading all assets, creating the user interface, and starting the WebXR session. During initialization, the interactive meshes representing stars and negative objects are registered as grabbable

elements. Instead of creating separate object classes, each mesh is extended with metadata that stores its interaction type, active state, and original scale. This approach keeps the implementation lightweight and allows imported models to be integrated without restructuring their hierarchy.

All core gameplay functionality is implemented in the module `vr-grab-system.js`. This module manages the complete game state, including starting, pausing, resetting, and updating the timer, and it also contains the central picking and grabbing logic. Device-specific input handling is deliberately excluded from this module in order to guarantee that the behavior of the game remains identical across platforms. Platform-dependent functionality is implemented in separate modules, most notably `vr-grab-visionpro.js`, which maps hand tracking, pinch gestures, and gaze interaction to the shared grab system.

This separation ensures that the same interaction pipeline is executed regardless of whether the input originates from a controller, a hand gesture, or a gaze-based selection. As a result, differences in performance or usability can be attributed to the hardware and tracking system rather than to variations in the application logic.

B. Picking and Grabbing Pipeline

Interaction with virtual objects is based on raycasting. For each active input source, a ray is generated and tested against the scene using Babylon.js' picking system. The origin and direction of this ray depend on the available tracking data. If a pointer transform is available, it is used directly. Otherwise, the system derives a forward vector from the grip pose of the controller or hand.

Because imported models often consist of multiple nested meshes, the initially picked mesh is resolved to the nearest parent that has been registered as grabbable. Once a valid target has been identified, the unified interaction handler processes the grab event. This handler performs several validation steps, including checking whether the game is currently active, ensuring that the object has not already been collected, and applying a per-controller debounce interval of 500 ms to prevent multiple triggers from a single physical action.

For collectible stars, the interaction is completed immediately upon a successful grab. The score is increased, the object is marked as inactive, and visual and audio feedback is triggered. This design choice guarantees that the score reflects the interaction without delay and makes timing measurements unambiguous. For other interactive objects, the system initiates a hold operation in which the object follows the hand or controller until the grab is released.

C. Hold-to-Move Interaction

The hold mechanism is implemented by temporarily detaching the selected object from its original parent and updating its position on every frame so that it follows the tracked input source. During this phase, recent positions are sampled to estimate a release velocity, which can be used to create a simple throwing effect when the object is released.

When the grab ends, the per-frame observer responsible for updating the object's position is removed and the final interaction is evaluated. Before applying scoring or effects, the system verifies that the object is still active. This prevents double counting in situations where the initial grab has already consumed the object.

This distinction between immediate collection and continuous holding allows the interaction model to remain responsive for fast, repetitive tasks while still supporting more complex manipulation when required.

D. Meta Quest Controller Integration

On the Meta Quest platform, controller input is accessed through the WebXR gamepad interface. When a controller is detected, its button state is polled in every render loop iteration. A rising-edge detection is used to identify the moment when the user presses the grab button. This polling-based approach ensures consistent behavior across runtimes and avoids timing issues that can occur with event-based input.

When a button press is detected, the shared picking pipeline is executed and the selected object is processed immediately. The high positional stability of the controller results in precise raycasting and allows rapid repeated selections with minimal error.

E. Apple Vision Pro Hand Tracking and Gaze Interaction

On Apple Vision Pro, interaction is based on the combination of gaze targeting and pinch gestures. The primary implementation uses the standard WebXR `selectstart` and `selectend` events, which are generated when the user performs a pinch. These events are forwarded directly to the unified grabbing system, ensuring that the same interaction logic is executed as for controller input.

Since some runtimes do not reliably map pinch gestures to select events, a fallback mechanism based on joint-distance measurement was implemented. In each XR frame, the positions of the thumb tip and index finger tip are retrieved through the WebXR Hand API. The Euclidean distance between these joints is calculated, and a pinch is detected when this distance falls below a configurable threshold. When the distance increases again, the grab is released. This method allows pinch detection even when the runtime does not emit explicit selection events.

Gaze-based targeting is implemented by performing a raycast from the XR camera in every frame. When the ray intersects a grabbable object, a highlight effect is applied by slightly scaling the object and modifying its emissive color. This visual feedback is essential for confirming the current target before the user performs the pinch gesture.

F. Feedback and Effects

To provide immediate confirmation of successful interactions, each grab triggers a short-lived visual effect at the interaction position. These effects are generated procedurally and automatically disposed after a short animation in order to minimize memory usage and maintain stable performance.

Audio feedback is played simultaneously, reinforcing the perception of responsiveness and improving the overall user experience.

G. Platform Constraints and Robustness

Because WebXR immersive sessions and the WebXR Hand API require a secure context, the application must be served over HTTPS during testing. The implementation is designed to degrade gracefully when certain features are not available. If hand joint data cannot be accessed, the system relies entirely on select events for pinch detection. Similarly, the platform detection logic is kept conservative to avoid incorrect initialization paths.

Overall, the implementation demonstrates that a strict separation between interaction logic and platform-specific input mapping enables a single WebXR application to support multiple XR devices while maintaining consistent gameplay behavior. This architecture simplifies testing, ensures comparability of performance measurements, and provides a flexible foundation for future extensions such as multi-user interaction and automated metric logging.

V. Performance Measurement Framework

To assess the feasibility of cross-platform, browser-based XR applications, this project has designed a performance evaluation framework. This framework is used not only to evaluate rendering performance but also to analyze interaction quality, system responsiveness, and user experience across different XR devices. Since this project aims to compare the Meta Quest and Apple Vision Pro within the same WebXR application, the framework focuses on metrics that reflect both technical performance and user experience.

This evaluation is based on five key dimensions: frame rate stability, input latency, interaction accuracy, visual feedback quality, and overall gaming experience. Frame rate stability measures whether the application can maintain a smooth, immersive experience during real-time interactions. Input latency refers to the perceived delay between user actions (such as pressing a controller button or performing a pinch gesture) and the system's corresponding response (including score updates, sound playback, or visual effects). Interaction accuracy measures the reliability with which target objects can be selected in each interaction mode. Visual feedback quality assesses whether effects such as highlighting, particle effects, and object disappearance are clear and behave as expected. Game performance is reflected by the score achieved within a fixed round duration and serves as an overall indicator of the actual efficiency of interaction.

To ensure comparability, both devices used the same game scenarios, object distributions, and round durations. Each round lasted 60 seconds, during which users attempted to collect stars and avoid bombs. Multiple rounds were conducted on Meta Quest to obtain a representative average score. On Apple Vision Pro, however, the grabbing interaction could not be fully implemented successfully. As a result, some performance dimensions, especially successful object collection and interaction timing, could not be measured in the same way as on Meta Quest. Instead, the evaluation for Apple Vision Pro was limited to observable performance indicators such as frame rate, gaze target stability, and the responsiveness of the selection interface.

VI. Results

A. Interaction Performance

On Meta Quest, the interaction system was successfully implemented and fully functional. Users were able to reliably select and collect objects using controller-based input. We conducted five rounds of testing, with scores of 47, 52, 50, 50, and 53, respectively, indicating stable performance and a relatively high selection accuracy rate.

However, the visual feedback associated with object pickup did not behave as expected. The particle-like scattering effect appeared visually blurred and did not fully dissipate over time. This suggests that the effect implementation may not be fully optimized or correctly configured for the rendering pipeline, although it did not significantly impact the core gameplay functionality.

In contrast, the grabbing system on Apple Vision Pro could not be successfully realized. Although gaze-based targeting and pinch gesture detection were partially implemented, performing a pinch gesture did not trigger object interaction. As a result, it was not possible to measure interaction success rate, reaction time, or gameplay score on this platform.

To compensate for this limitation, a debugging mechanism was introduced that displays the identifier of the currently gazed object in real time. This allowed partial evaluation of the gaze interaction system. Observations showed that gaze locking was unstable, with frequent switching between nearby objects, leading to a lack of precision and reduced intuitiveness.

B. System Performance

Both devices maintained acceptable frame rate performance during testing. On Meta Quest, the frame rate ranged between 30 and 40 frames per second, providing a generally smooth experience during gameplay. On Apple Vision Pro, the frame rate was slightly higher, ranging between 40 and 50 frames per second.

In addition to rendering performance, the application included visual and audio feedback after successful object collection. On Meta Quest 3, the visual effect did not behave as intended. Instead of showing a clear scattering-and-fading animation, the particle-like effect appeared blurred and rigid, and the expected spreading and disappearing behavior was not successfully achieved. On Apple Vision Pro, this part could not be validated because object grabbing was not successfully implemented, and therefore, the corresponding effects could not be triggered in a complete interaction flow.

These results indicate that the simplified scene design effectively reduced rendering complexity and allowed both devices to maintain stable runtime performance. However, they

also suggest that a stable frame rate alone is not sufficient to guarantee satisfactory feedback quality, as visual effects may still require further adjustment and platform-specific optimization.

C. Interaction Stability and Usability

User experience differed significantly between the two platforms. On Meta Quest, controller-based interaction provided stable targeting and consistent feedback, resulting in high user confidence and ease of use. The physical controller and direct input mapping contributed to a predictable and responsive interaction experience.

On Apple Vision Pro, the gaze-based interaction introduced challenges in stability and intuitiveness. The lack of reliable grab triggering, combined with unstable gaze locking, made it difficult for users to perform precise selections. Even though gaze-based interaction is conceptually natural, its current implementation in this system did not provide sufficient accuracy for fast-paced interaction tasks.

VII. Discussion

The results of this project show that WebXR can provide a shared technical basis for deploying the same XR application on different devices. The application was successfully tested on Meta Quest, where the core gameplay and interaction loop worked as intended. This demonstrates that the overall architecture of the system is feasible for browser-based XR development.

However, the experiment was not fully completed on both platforms. In particular, the grabbing interaction on Apple Vision Pro could not be successfully implemented in the code developed for this project. Although gaze-based targeting and pinch-related input detection were partially realized, object collection could not be triggered reliably. As a result, the comparison between Meta Quest and Apple Vision Pro remains incomplete, and the evaluation of Apple Vision Pro is limited to partial observations, such as frame rate and gaze target stability.

This study also has several limitations. First, the correctness and precision of the implementation directly affected the outcome, especially on Apple Vision Pro. Second, the experiment involved only one participant so that the observations may reflect individual usage behavior rather than general user performance. Third, the study's data basis is limited, as the conclusions were drawn from a small number of test runs and a single primary data source. Therefore, the findings should be interpreted as exploratory rather than conclusive.

Overall, the project confirms the potential of WebXR for cross-platform XR development, but it also shows that reliable cross-device interaction cannot be assumed automatically. Future work should focus on completing the Apple Vision Pro grabbing logic, improving interaction robustness, and conducting a broader evaluation with more participants and more systematic data collection.

VIII. Conclusion

This project shows that WebXR can be used as a common basis for developing browser-based XR applications. The system was successfully implemented and tested on Meta Quest, which confirms the feasibility of the proposed architecture in practice. However, the Apple Vision Pro interaction logic could not be fully completed in the current implementation, so the cross-device evaluation remains partial. Overall, the project provides an initial practical exploration of WebXR-based cross-device XR development.

Acknowledgment

We would like to thank our supervisor, Louay Bassbouss, for the guidance and encouragement throughout this project. We are also grateful for the opportunity to use the experimental devices, which enabled the practical implementation and testing of the system.